Лабораторная работа № 3. Экранный доступ в MacOS: от навигации к Accessibility API

Зафиксированная среда:

- **Модель и процессор:** Виртуальная машина macOS (Архитектура: Intel x86_64);
- **Версия и сборка macOS:** macOS Catalina 10.15.5 (Сборка: 19F101);
- **Браузер:** Firefox 159.0;
- **Версия Xcode:** Xcode 12.4 (Build 12D4e);
- **Скринридер:** Встроенный Apple VoiceOver (в составе macOS 10.15.5);
- **Язык и синтезатор речи:** Русский, системный голос «Милена»
- **Модификатор VoiceOver (VO):** Control + Option или Caps Lock;

Часть 1. Базовая невизуальная навигация с VoiceOver и архитектура экранного доступа в macOS

Для выполнения работы был выбран сайт Яндекс Погода. Главный экран сайта показан на рисунке 1.

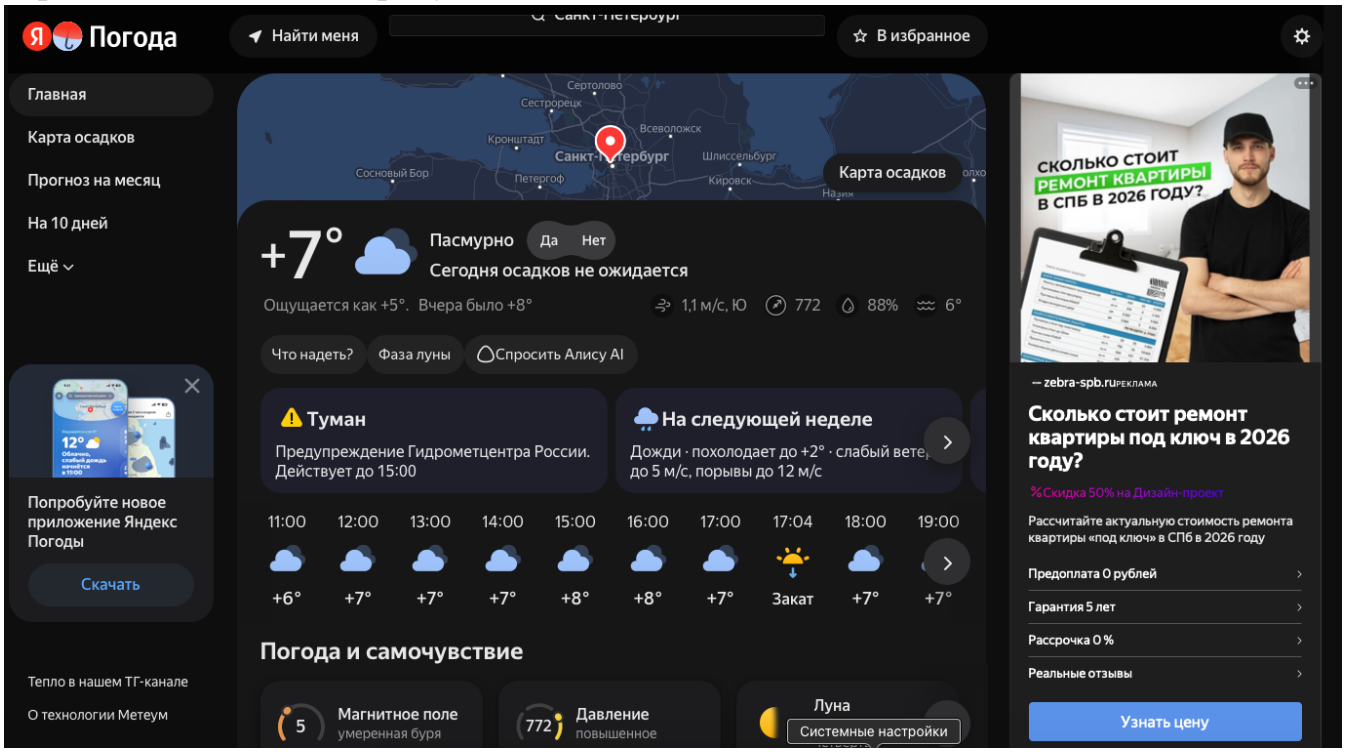


Рисунок 1 - Главная страница Яндекс Погода

Перемещения производились от вкладки “Главная”.

Протокол действий:

№	Участник	Что хотел сделать	Как перемещался	Что произнес VoiceOver	Получилось	Комментарий
1	Колитченкова П.С.	Найти кнопку определения геопозиции	Tab, 2 раза	“Найти меня, кнопка”	Да	Перед кнопкой главная
2	Колитченкова П.С.	Найти строку поиска	Tab, 3 раза	“найти город или район, поле поискового запроса”	Да	Озвучивает содержимое поля
3	Колитченкова П.С.	Найти и раскрыть меню настроек	Tab, 10 раз, Enter, 1 раз	“Меню настроек, всплывающее меню свернутая, кнопка”	Да	При открытии меню 1 раз повторяет содержимое меню
4	Колитченкова П.С.	Переход по ссылке в справку	Tab, 10 раз, Enter, 1 раз, Tab, 8 раз, Enter, 1 раз	“Веб-контент”	Да	При переходе по ссылке озвучивает все элементы на странице
5	Колитченкова П.С.	Ввести текст в поле ввода и/или выбрать город	Tab, 6 раз, Enter, 1 раз	“ссылка, Москва Объектов: список Список истории поиска и других городов 6”	Да	После строки поиска озвучивает две строки «табуляции» с озвучиванием текущего города

Таблица объектов:

Объект	Что видно визуально	Что говорил VoiceOver	Что говорил NVDA	Роль	Имя	Состояние
Найти меня	Значок в виде стрелки и надпись “найти меня”	“Найти меня кнопка”	“Найти меня кнопка”	кнопка	Найти меня (AXTitle, AXDescription)	AXEnabled, AXFocused
В избранное	Иконка звезды и надпись “в избранное”	“В избранное, кнопка, баннер”	“В избранное, кнопка”	кнопка	В избранное (AXTitle)	AXEnabled, AXFocused
Справка	Знак вопроса в круге и надпись “Справка”	“ссылка, справка”	“Справка, ссылка”	Ссылка	Справка (AXTitle)	AXEnabled, AXFocused

Сравнение речевого и брайлевского вывода

Была исследована кнопка «найти меня». На рисунке 2 и 3 представлены брайлевские выводы из NVDA и VoiceOver соответственно.



Рисунок 2 – Брайлевский вывод NVDA



Рисунок 3 – Брайлевский вывод Orca

Вывод роли «кнопка» в VoiceOver происходит аналогично голосовому выводу в конце фразы. В NVDA «кнопка» находится в начале строки. В обеих системах используется стандартный компьютерный 8-точечный шрифт Брайля. Роль элемента («кнопка») передается отдельным символом в начале (в NVDA) или сокращением в конце (в VoiceOver), за которым следует имя элемента.

Архитектурная схема

На рисунке 4 представлена архитектурная схема.

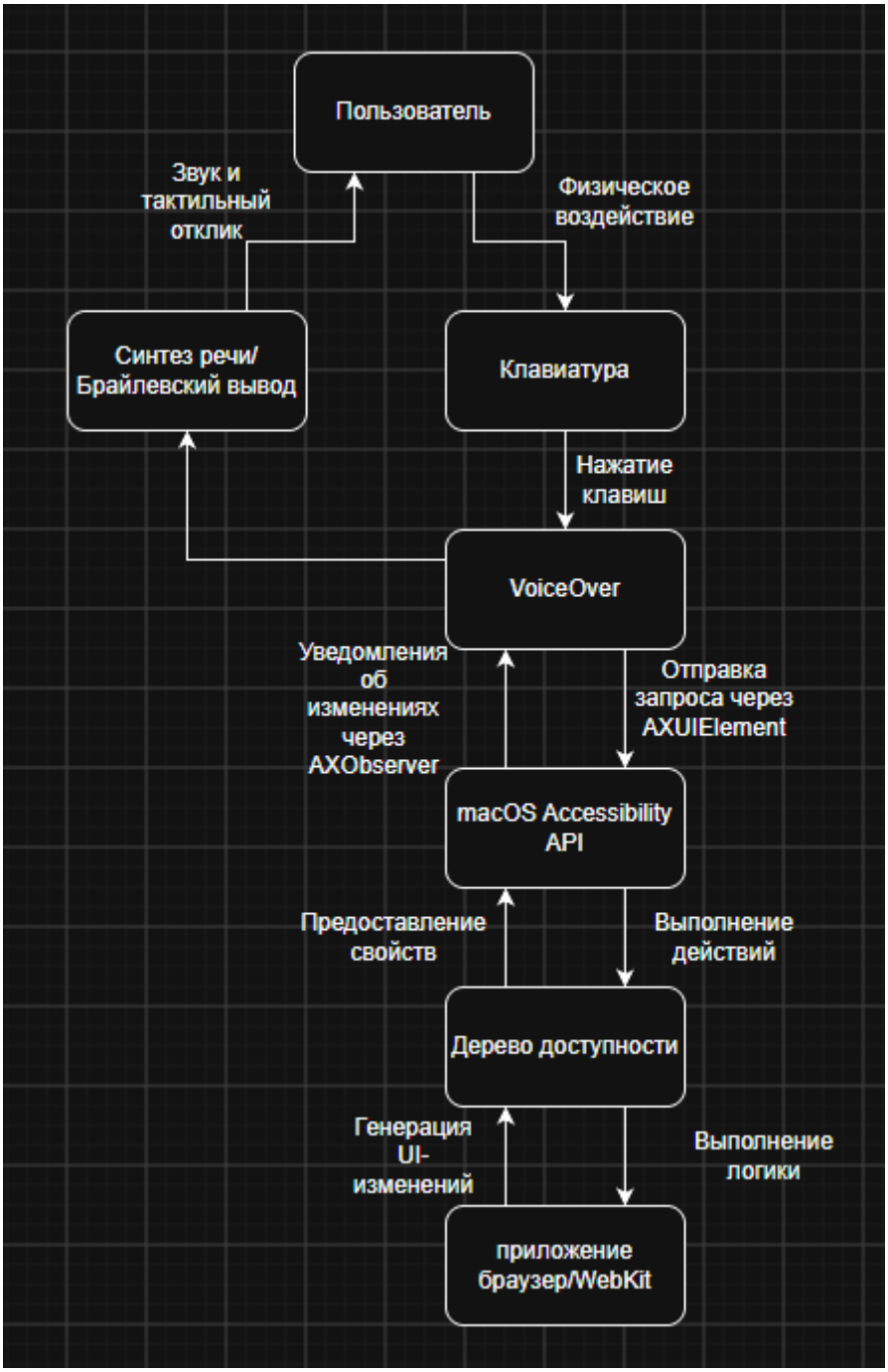


Рисунок 4 – архитектурная схема в macOS

Часть 2. Исследование доступности приложения macOS через Accessibility API

Была сделана html-страница, представленная на рисунке 5. Аналогичная страница была использована в предыдущей работе.

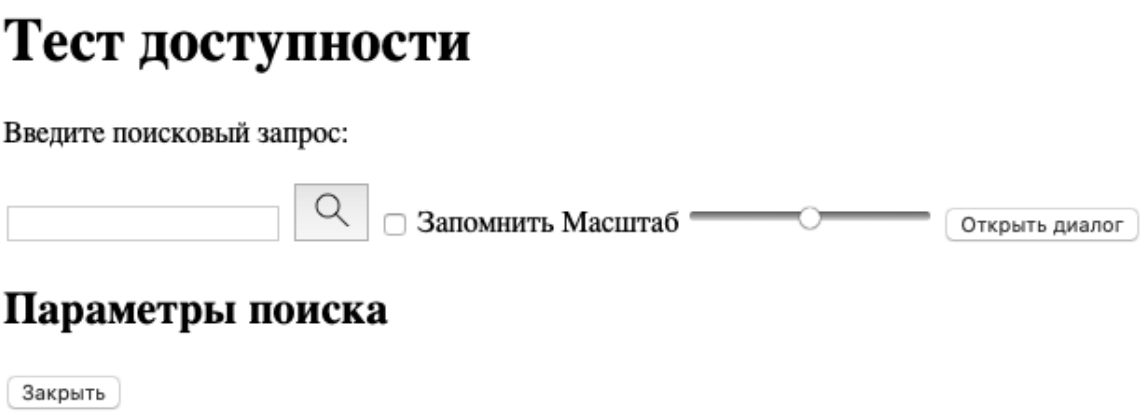


Рисунок 5 – Html-страница для задания

Инспекция

Элемент	Роль/подроль	Заголовок, описание, подпись	Значение	Состояния	Доступные действия	Родитель в дереве
Поле ввода	ATXTextField	- (пусто)	пусто	AXEnabled, AXFocused	Показать меню (AXShowMenu), AXScrollToVisible	Группа
Кнопка поиска	AXButton	- (пусто)	-	AXEnabled, AXFocused	AXPress, AXShowMenu, AXScrolltoVisible	Группа
Флажок запомнить	AXCheckBox	Запомнить	0	AXEnabled, AXFocused	AXPress, AXShowMenu, AXScrolltoVisible	Группа
Ползунок Масштаб	AXSlider	Масштаб	50	AXEnabled, AXFocused	AXShowMenu, AXScrolltoVisible, AXIncrement, AXDecrement	Группа
Кнопка	AXButton	Открыть диалог	-	AXEnabled, AXFocused	AXPress, AXShowMenu, AXScrolltoVisible	Группа

Журнал событий

№	Действие пользователя	Уведомление AX	Источник	Что сказал VoiceOver	Аналогичное событие в UI Automation
1	Нажать Tab для перехода к полю ввода	AXFocusedUIElementChanged	AXTextField	“Можно редактировать текст пустой”	AutomationfocusChanged
2	Нажать стрелку вправо/влево на ползунке «Масштаб»	AXValueChanged	AXSlider («Масштаб»)	50, Масштаб, бегунок	PropertyChanged для RangeValue.Value
3	Открыть окно	AXFocusedUIElementChanged Было открыто новое окно Safari	AxWindow	Safari, избранное, окно, адрес и поиск…	StructureChanged

Барьеры

Кнопка поиска имеет только графический символ лупы с атрибутом aria-hidden=”true”, отсутствует содержимое и нет имени -> VoiceOver сообщает только “кнопка” -> пользователь может не понять значения этой кнопки и не понимает как отправить поисковый запрос -> воспроизводится по Tab из поля ввода на соседнюю кнопку.

Поле ввода поискового запроса -> Визуальная подпись «Введите поисковый запрос:» оформлена тегом <p> и программно не связана с полем ввода через <label> или aria-label. Свойство Name не сформировано -> скринридер сообщает только “можно редактировать текст пустой” -> пользователь не понимает, что необходимо вводить в поле -> воспроизводится нажатием клавиши Tab для установки фокуса в поле ввода.

Исправление барьеров

Был исправлен барьер кнопки поиска. Фрагмент кода до представлен в листинге 1, фрагмент кода после - в листинге 2.

Листинг 1 - Фрагмент кода с барьером до исправления

```
<button id="search" type="button">
  <svg aria-hidden="true" ...> ... </svg>
</button>
```

Листинг 2 - Фрагмент кода с барьером после исправления

```
<button id="search" type="button" aria-label="Поиск">
  <svg aria-hidden="true" ...> ... </svg>
</button>
```

Негативный тест:

- **Сценарий:** Пользователь нажимает Tab (или использует навигацию VoiceOver) для перехода на кнопку поиска в исходной версии страницы.
- **Показания инспектора (Accessibility Inspector):** Узел имеет роль AXButton. Свойства AXTitle и AXDescription пустые (свойство Name не сформировано).
- **Поведение VoiceOver:** «кнопка».
- **Результат теста:** Пользователь не понимает, что делает эта кнопка, и не может уверенно воспользоваться поиском без зрительного контроля.

Позитивный тест:

- **Сценарий:** Пользователь переходит на кнопку поиска после добавления атрибута aria-label.
- **Показания инспектора (Accessibility Inspector):** Узел имеет роль AXButton. В свойствах появляется значение AXDescription (или AXTitle) = «Поиск».
- **Поведение VoiceOver:** «Поиск, кнопка».
- **Результат теста:** Назначение элемента передано полностью. Пользователь понимает, что нажатие этой кнопки инициирует поиск.

Сравнение до и после

До исправлений VoiceOver произносит только «кнопка», в Inspector имя пустое. После исправления VoiceOver произнес «Поиск, кнопка». При этом в Inspector отобразилось имя «Поиск».

Итоговый вывод

В ходе выполнения лабораторной работы было установлено, что VoiceOver в macOS, как и NVDA в Windows, получает информацию об интерфейсе не через графический анализ, а через программный интерфейс Accessibility API, предоставляющий доступ к дереву объектов. При сравнении одних и тех же элементов (кнопок и полей ввода) выяснилось, что оба скринридера передают схожую семантику (роль, имя, состояние), хотя синтаксис фраз может отличаться из-за платформенных особенностей (например, VoiceOver часто меняет порядок озвучивания роли и имени). Общей для двух систем остается базовая модель экранного доступа: наличие клиента (скринридера), поставщика (приложения), а также механизмов запроса свойств (IUIAutomationElement в Windows и AXUIElement в macOS) и подписки на события через наблюдатели. Платформенные различия проявились в названиях событий (например, AutomationFocusChanged в UIA против AXFocusedUIElementChanged в AX API) и в реализации многопроцессной архитектуры браузеров. Важным отличием macOS является разделение системного фокуса ввода и курсора VoiceOver, которые могут находиться на разных элементах и управляются разными настройками. В ходе работы были выявлены ограничения инструментов: на macOS Catalina Safari блокировал локальные скрипты до снятия ограничений, а утилита AXObserver не зафиксировала событие открытия HTML-диалога из-за изоляции процессов WebKit. Таким образом, цифровая доступность в обеих ОС зависит от корректности программного дерева, а выявление и исправление барьеров (например, добавление <label> или aria-label) делает интерфейсы универсальными для невизуального использования.